ROS2 자율주행 프레임워크에서의 수직 주차 성능 향상

김수경 [1], 강민철 [1], 김명균 [2], 박효진 [3], 전재욱 [4]
[1] 성균관대학교 기계공학부 학부생
[2] 성균관대학교 전자전기공학부 학부생
[3] 성균관대학교 전자전기컴퓨터공학과 석사과정
[4] 성균관대학교 전자전기컴퓨터공학과 교수

ksk192830@g.skku.edu, kmc6042@g.skku.edu, mk11226@g.skku.edu, phyojin0511@g.skku.edu, jwjeon@skku.edu

Enhancing Perpendicular Parking in a ROS2 Autonomous Driving Framework: A State-Based Multi-Sensor Approach

Sugyeong Kim[1], Mincheol kang[1], Myungkyun Kim[2], Hyo Jin Park[3], Jae Wook Jeon[3]
[1] School of Mechanical Engineering, Sungkyunkwan University
[2]Department of Electronic and Electrical Engineering, Sungkyunkwan University
[3]Department of Electrical and Computer Engineering, Sungkyunkwan University

Abstract

This paper describes the implementation of a vertical parking function based on the existing modular ROS2 autonomous driving framework. To increase accuracy compared to the existing module, the pipeline was redesigned using a phase-aware perception-decision-control approach. This combines LiDAR-based parking area search with front camera and rear camera and the analysis of a bird's-eye view. A finite-state controller manages parking area search, alignment, reverse entry, stopping, and exit. Verification through analysis of nominal, boundary, and failure-oriented scenarios resulted in improved parking consistency, and parking performance was enhanced while retaining the existing modular framework structure.

1. Introduction

Parking a car into a perpendicular slot may seem simple, but automating the maneuver is one of the more challenging tasks in an autonomous driving project. The car needs to figure out where an empty slot is, reverse into it without hitting anything, and end up reasonably straight. Each of those steps pulls in different sensors and different control logic.

Camera-based autonomous parking is widely used on real cars which use segmentation and BEV projection. In previous project, [1] we implemented lane keeping, traffic sign detection, dynamic/static obstacle avoidance, and perpendicular parking. However, the perpendicular parking accuracy was significantly low. So, we put away the previous LiDAR-based parking module and newly developed a camera-based parking module. Where the earlier work [1] dealt with general road driving, this paper is specifically about parking.

Our contributions are as follows. (1) A four-phase state machine (slot searching, reverse entry, posture adjustment, exit) where each phase activates only the sensors it needs—this was necessary because our laptop CPU maxed out at around 92% if we ran everything at once. (2) Two YOLO11n-seg models trained on different class sets for the front and rear cameras respectively. (3) A geometry pipeline that takes BEV-warped segmentation masks and produces parking lines, center lines, and end lines through morphological cleanup, contour simplification, and line fitting.

2. Parking System Design

The parking operation runs through four phases in sequence. Each one has a different job and uses different sensors.

2.1. Autonomous parking system architecture

Slot Searching. The car drives slowly forward alongside a row of parked vehicles. The front camera segments the *parking_lot* boundary, and from the BEV-warped mask we extract a line that tells the controller how far the car is from the row. At the same time the 2D LiDAR watches for gaps between parked cars. When the LiDAR range readings jump beyond a threshold (we used 2.0 m) for a sustained interval, the state machine flags that gap as a candidate slot and moves on.

Reverse Entry. The car begins to move backwards toward the identified slot. The rear detection model segments the *parking_space* region, and the post-processing code computes a center line from the mask edges. The steering

controller tracks this center line to guide the car into the slot.

Posture Adjustment. The rear camera provides midline data of the slot and the car does small angle adjustments until the heading error drops below a threshold. When the rear camera detects the *end_line* at the purposed location, adjustment phase terminates.

Exit. The car pulls forward out of the slot. The front camera picks up an *out_line* marking ahead, and the controller aligns the car's heading with that line before handing control back to the main driving stack.

Phase transitions use a combination of LiDAR triggers and flags from the vision nodes. The current phase is published on the */motion_state* topic (*std_msgs*/String), and every perception node listens to it.

2.2. Selective perception node activation

All of our system (two YOLO models, the BEV geometry nodes, and the motion controller) run on a single laptop. To make the system light and reliable each YOLO node reads the */motion_state* topic and checks an *allowed_states* list in its launch config. If the current phase is not in the list, the node skips the inference entirely.

2.3. ROS2 Node Architecture

Parking Front/Rear Detection: These nodes process segmentation outputs from the front and rear camera images, apply BEV transformation and extract geometry-based features required for parking. Then, the extracted features are published as corresponding messages.

Motion Parking: The node consumes the outputs of both detection nodes (Front Parking Line, Front Out Line, Rear Parking Space) to compute steering commands and then utilize Rear End Line and LiDAR proximity data to determine stopping points and state transition timing.

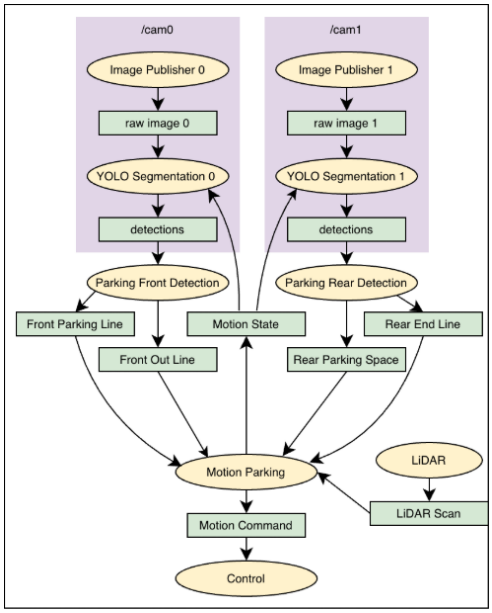


Fig. 1 Node graph and topic flow for the parking subsystem.

3. Perception Pipeline

3.1. YOLO Segmentation Models

Two of segmentation models that trained with YOLO11n-seg were trained for the front camera and rear camera views, reflecting different semantic classes required by each parking step.

Front model classes: *parking_slot* (the individual slot markings), *parking_lot* (the overall parking area boundary), and *out_line* (the exit-direction reference line).

Rear model classes: *parking_slot*, *parking_lot*, and *end_line* (the rear boundary of the parking slot).

The Nano variant was selected for its ability to run at adequate frame rates in case of a CPU-only deployment. Both models were trained utilizing the Ultralytics YOLO framework with a pretrained yolo11n-seg.pt backbone.

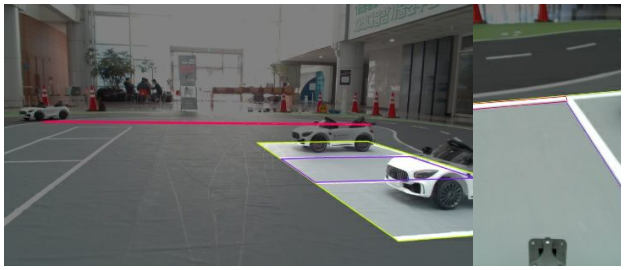


Fig. 2 Labeled data of front view(left) and rear view(right)

3.2. Training configuration

Table 1. Dataset and Training Configuration

	Front Model	Rear Model
Base Architecture	YOLO11n-seg	YOLO11n-seg
Original Images	4,466	2,350
Augmented Images	10,720	5,640
Epochs	10	10
Batch Size	Auto	Auto
Device	GPU (CUDA:0)	GPU (CUDA:0)

We made a data augmentation in Roboflow because of the increasing a training set that has more noises and difference rotation and shear. The augmentation pipeline included random rotation (±15°), shear (±10° horizontal and vertical), and pixel noise (up to 1.01% of pixels). These transformations simulate variations in camera angle and lighting that occur during actual parking steps.

3.3. Training Results

Table 2. Final Validation Metrics (Epoch 10)

Metric	Front Model	Rear Model
Precision (Box)	0.892	0.968
Recall (Box)	0.921	0.968
mAP50 (Box)	0.924	0.985
mAP50-95 (Box)	0.764	0.888
Precision (Mask)	0.838	0.932
Recall (Mask)	0.866	0.924
mAP50 (Mask)	0.871	0.932

mAP50-95 (Mask)	0.656	0.787
Val Box Loss	0.726	0.452
Val Seg Loss	1.125	0.631
Val Cls Loss	0.519	0.368

The rear model beats the front model across the board. We thought the rear view was simpler and more consistent and the front view was more complex and has a lot of dynamic scenes. So, for these reasons, this situation occurred.

We used 10 epochs, because of our environment has limited GPU performance. The front model showed a noticeable train–validation gap (1.125 vs. 0.968), indicating incomplete convergence, while the rear model converged more stably. Additional training (approximately 20–25 epochs) is expected to further improve the front model.

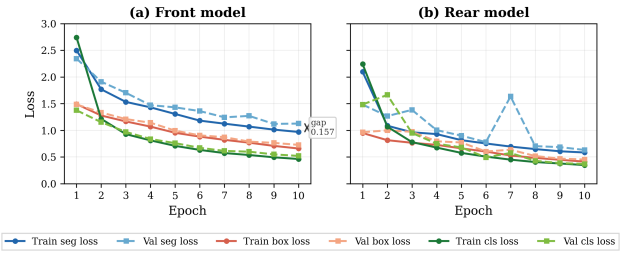


Fig. 3 Training & validation loss curves for the front (a) and rear (b) YOLO11n-seg models. Solid lines denote training loss; dashed lines denote validation loss.

4. BEV-Based Geometric Processing

4.1. Mask Post-Processing Pipeline

Raw segmentation masks from the YOLO segmentation models are well known to be noisy. To complement this problem, the system pre-processes the mask for clear results.

Hole Filling: A flood fill operation starting from the image corner fills internal holes in the mask.

Connected Component Filtering: For the *parking_lot* class, only the N largest connected components (default N=2) are retained, eliminating the noise.

Polygon Simplification: The Douglas-Peucker algorithm is applied iteratively to reduce the point of each mask contour to a simplified polygon with 6–8 vertices.

Mask Union and Refinement: The *parking_lot* and *parking_space* masks are merged using a morphological close operation (kernel 13×13) producing a unified parking-region mask.

4.2. Control Feature Extraction

Front Parking Line: Extracted by fitting a line to the left boundary of the parking-lot mask in BEV space, used for lateral offset control.

Front Out Line: Obtained by fitting a line to the extracted stripe region of the *out_line* mask, providing the exit direction.

Rear Parking Space Center Line: Computed from detected parking-space boundaries using edge and line detection, enabling reverse alignment.

Rear End Line: Derived by fitting a line to the *end_line* mask, used for stopping-point determination.

5. Experimental validation

5.1. Test Environment

The auto parking system was tested and qualified on the autonomous driving competition course at the Gyeonggi-do autonomous driving test track. The vehicle is equipped with a front camera, a rear camera, and a 2D LiDAR. All computation was performed on a single notebook computer running Ubuntu with ROS2.

In most case of real-time rollouts, the vehicle successively completed all parking phases.

5.2. Failure Cases and Observed Limitations

LiDAR-Based Transition Failure: In some cases, the LiDAR failed to detect vehicles, causing the system to miss the vacant slot. This was attributed to the unreliable power supply to the 2D LiDAR and accumulated cache from repetitive operations.

Rear-Camera Detection Failure: The rear camera failed to detect the *parking_space* when observing unseen lighting conditions with respect to the collected data. The rear parking space detection node malfunctioned resulting in parking task failure. The center-line computation is particularly more sensitive to this change since the logic required various conditions. Lowering the frame rate reduced this malfunction and made the process smoother.

Insufficient Steering During Alignment: In some posture adjustment situations, the maximum steering angle was insufficient to fully align the vehicle with the target parking lines, resulting in the vehicle to partially touch the adjacent slot boundaries.

6. Conclusion

This paper describes the implementation of a vertical parking function using ROS2. Accuracy was improved by dividing the process into four steps. Additionally, the YOLO11n-seg model was used to represent front and rear camera images using a segmentation mask that has undergone BEV conversion.

Through this research, it was confirmed that the vertical parking function can be used in real-world situations, and it was also possible to identify situations where accuracy drops or failure is possible.

Acknowledgements

This work was supported by the Korea Institute for Advancement of Technology (KIAT) grant funded by the Ministry of Education (Project No. P0030064, Advanced Industry Talent Training Bootcamp Program – Sungkyunkwan University (Future Mobility))

References

[1] S. Kim, Y. Woo, and K. Kim, "Development of a Modular ROS2 Software Framework for Autonomous Vehicles," in *Proceedings of the 2025 KSAE Annual*

Autumn Conference and Exhibition, Busan Port International Exhibition & Convention Center (BPEX), pp. 1896–1903, 2025.

[2] J. Lee, M. Kim, J. Jung, J. Joo, H. Choi, J. Lee, and J. Jeon, "Implementation of Control Systems and Algorithms for Autonomous Vehicles," in *Proceedings of the 2024 KSAE Annual Autumn Conference*, pp. 2948–2954, 2024.